

CEB-GNRD OpenBMC 快速上手与开发指南 (Quick Start Guide)

本文档详细说明如何从零开始构建 `ceb-gnrd` 固件镜像，如何在 QEMU 模拟器中启动并测试 WebUI/SSH/Redfish，以及如何遵循 OpenBMC 开发规范、使用 `devtool` 进行源码修改、新增代码与板级 Patch 产出。

硬件平台说明：

本项目面向 **Intel Xeon 6** 单路主板平台，BMC 芯片为 **ASPEED AST2600**。

QEMU 模拟环境主要用于验证 OpenBMC 用户空间、`bmcweb`、WebUI 前端、SSH、网络配置及核心 D-Bus 接口流程，不能替代真实硬件环境中的 eSPI、PECI、ADC、PMBus、PWM/TACH 和实体网口测试。

参考资料与推荐教程

- 官方/社区核心教程：[Home | OpenBMC Guide Tutorial](#)
- OpenBMC 官方文档：[OpenBMC Documentation](#)
- Yocto Devtool 官方参考手册：[Yocto Application Development and the Extensible SDK](#)
- OpenBMC WebUI 前端仓库：[webui-vue](#)

一、OpenBMC 开发与分层规范原则

在 OpenBMC 开发中，严格遵循分层隔离与代码收敛原则，能确保后续平滑同步社区上游更新：

1. 板级目录隔离原则 (Board-Level Layering)：

- 严禁直接修改上游图层（如 `upstream-layers/openembedded-core`、`meta-phosphor`、`meta-aspeed`、`meta-openembedded` 等）。
- 所有针对本板卡的修改（设备树 DTS、Entity-Manager JSON、IPMI YAML、systemd 网卡配置、自定义服务脚本、上游软件功能补丁）**必须全部集中收敛在板卡级目录**：`meta-ctopai/meta-ceb-gnrd/`。

2. Patch 与 bbappend 管理规范：

- 对任何开源组件（如 Linux 内核、`bmcweb`、`webui-vue`、`dbus-sensors` 等）的修改与新增代码，均应通过板卡层中的 `.bbappend` + `.patch` 补丁方式引入。
- 补丁文件存放在对应配方目录的 `files/` 下，使用 `SRC_URI:append` 引入。
- 补丁必须具备清晰的 Git Commit Title、功能说明和 `Signed-off-by` 签名。

3. 配置与接口解耦 (D-Bus / Redfish 标准化)：

- 硬件拓扑与传感器接入优先通过 Entity-Manager JSON 配置，利用 `dbus-sensors` 自动加载，避免在底层驱动或服务中硬编码物理 I2C 地址。

二、构建环境准备

以下命令应在 Linux 主机或虚拟机中执行（推荐使用 **Ubuntu 22.04 LTS / 24.04 LTS**）。

1. 安装 OpenEmbedded / Yocto 基础依赖

```
sudo apt update
sudo apt install -y \
    build-essential chrpath diffstat gawk git wget \
    libegl1-mesa libstdc++12-dev pylint3 python3 python3-git \
    python3-jinja2 python3-pexpect python3-subunit socat \
    texinfo unzip xterm zstd file locales
```

2. 设置 UTF-8 语言环境

```
sudo locale-gen en_US.UTF-8
export LANG=en_US.UTF-8
export LC_ALL=en_US.UTF-8
```

三、从零开始构建 ceb-gnrd

1. 获取并初始化源码仓库

```
git clone https://github.com/flether-hub/openbmc.git openbmc
cd openbmc
git submodule update --init --recursive
```

确认板级层结构完整：

```
test -f meta-ctopai/meta-ceb-gnrd/conf/machine/ceb-gnrd.conf && \
    echo "ceb-gnrd machine layer found"
```

2. 初始化构建环境与配置

在 OpenBMC 开发中，通常有两种初始化环境的方式：

方式 A：使用 `. setup ceb-gnrd`（推荐：用于全新创建与初始化）

```
cd ~/openbmc
. setup ceb-gnrd
```

- **作用：** OpenBMC 定制的引导配置脚本。它会自动创建 `build/ceb-gnrd` 目录，生成包含正确板卡配置的 `conf/local.conf`（自动注入 `MACHINE = "ceb-gnrd"`）以及完整的 `conf/bblayers.conf` 图层配置，并自动切换当前 Shell 至构建目录。
- **适用场景：** 首次构建、全新创建构建目录或需要重新生成标准构建配置时使用。

方式 B：使用 `source oe-init-build-env build/ceb-gnrd`（用于日常进入已有环境）

```
cd ~/openbmc
source oe-init-build-env build/ceb-gnrd
```

- **作用：** OpenEmbedded / Yocto 原生环境加载脚本。它主要负责配置当前 Shell 的 `PATH`、`BUILDDIR` 等 BitBake 运行时环境变量，并将工作路径切入 `build/ceb-gnrd`。它直接复用已有的配置文件，不会重新覆

写。

- 适用场景：日常开发、新开终端窗口或重新登录后继续编译已有构建目录时使用。

💡 两者核心区别对比速查

对比项	<code>. setup ceb-gnrd</code>	<code>source oe-init-build-env build/ceb-gnrd</code>
功能定位	OpenBMC 定制的一键初始化脚本	Yocto 原生环境变量配置脚本
配置文件生成	自动生成/覆写 <code>local.conf</code> 与 <code>bblayers.conf</code>	不生成新配置，直接复用现有配置文件
<code>MACHINE</code> 设定	自动在配置中锁定目标板卡为 <code>ceb-gnrd</code>	依赖已有配置或需手动声明 <code>export MACHINE=...</code>
主要使用时机	首次初始化 / 全新构建时使用	日常开发 / 新窗口恢复环境时使用

3. 构建 OpenBMC 镜像与关键目标

- 构建完整 BMC SPI Flash 固件镜像：

```
bitbake obmc-phosphor-image
```

- 单独编译 Linux 内核与设备树 (DTS)：

```
bitbake virtual/kernel
```

- 单独编译与验证板级配置包：

```
bitbake entity-manager dbus-sensors ceb-gnrd-hardware-contract
```

4. 构建产物说明

构建产物存放于：`build/ceb-gnrd/tmp/deploy/images/ceb-gnrd/`

- `obmc-phosphor-image-ceb-gnrd-*.static.mtd`：BMC SPI 原始固件镜像（供编程器烧录或 QEMU 启动）。
- `obmc-phosphor-image-ceb-gnrd-*.tar.gz`：BMC 根文件系统压缩包。
- `fitImage`：包含 Linux 内核与 Initramfs 的镜像。
- `aspeed-ceb-gnrd.dtb`：编译后的板级设备树二进制。

四、在 QEMU 虚拟机中启动与测试

1. 启动 QEMU 虚拟机

由于 CEB-GNRD 设备树将 MAC0/MAC1 禁用，并将 MAC2 映射为第一网口（`eth0`），启动时使用 `-net nic -net nic -net nic,netdev=net0` 将端口转发准确挂载到 MAC2：

```
ROOTFS=$(readlink -f tmp/deploy/images/ceb-gnrd/obmc-phosphor-image-ceb-gnrd-*.static.mtd | head -1)
qemu-system-arm -machine ast2600-evb -m 1G -nographic -drive file="$ROOTFS",if=mtd,format=raw -netdev user,id=net0,hostfwd=tcp:127.0.0.1:8443-:443,hostfwd=tcp:127.0.0.1:2222-:22 -net nic -net nic -net nic,netdev=net0 -serial mon:stdio -serial null
```

多行展开格式:

```
cd ~/openbmc
source oe-init-build-env build/ceb-gnrd
ROOTFS=$(readlink -f tmp/deploy/images/ceb-gnrd/obmc-phosphor-image-ceb-gnrd-*.static.mtd | head -1)

qemu-system-arm \
  -machine ast2600-evb \
  -m 1G \
  -nographic \
  -drive file="$ROOTFS",if=mtd,format=raw \
  -netdev user,id=net0,hostfwd=tcp:127.0.0.1:8443-:443,hostfwd=tcp:127.0.0.1:2222-:22 \
  -net nic -net nic -net nic,netdev=net0 \
  -serial mon:stdio \
  -serial null
```

2. 访问 OpenBMC 服务

- **Web 管理界面 (HTTPS):**

浏览器打开: <https://127.0.0.1:8443/#/>

(Chrome/Edge 提示“您的连接不是私密连接”时, 在键盘上直接盲打输入 [thisisunsafe](https://127.0.0.1:8443/#/) 即可进入登录页面)

- **SSH 登录:**

```
ssh -p 2222 root@127.0.0.1
```

- **Redfish API 接口测试:**

```
curl -k https://127.0.0.1:8443/redfish/v1/
```

3. 系统服务状态检查

在 BMC 终端中运行:

```
# 检查失败服务
systemctl --failed

# 检查 Web 后端与 Socket 监听
systemctl status bmcweb
systemctl status bmcweb.socket

# 检查实体管理服务与 IP 状态
systemctl status xyz.openbmc_project.EntityManager.service
ip addr
```

五、Devtool 常用操作与板级开发全流程

`devtool` 是 Yocto 推荐的源码级开发与补丁管理利器。使用 `devtool` 可以修改现有组件、向现有组件新增源代码文件、创建全新的独立软件包，并一键归档存入指定的板卡级图层。

分支 A：修改或向现有组件增加代码

1. 提取源码到工作区

```
devtool modify <recipe>
```



2. 修改/新增文件并提交

```
workspace/sources/<recipe>  
git commit -m "..."
```



3. 实时编译与验证

```
devtool build <recipe>
```



4. 自动生成 Patch 输出至板卡层

```
devtool finish <recipe> ../meta-ctopai/m  
eta-ceb-gnrd
```

分支 B：创建全新的独立软件包

1. 基于源码/仓库新建配方

```
devtool add <pkg> <src_path/url>
```



2. 自动生成配方并本地调试

```
workspace/recipes/<pkg>/  
devtool build <pkg>
```



3. 归档新配方至板卡层

```
devtool finish <pkg> ../meta-ctopai/meta  
-ceb-gnrd
```



4. 挂载至板卡镜像

在 `packagegroup-ceb-gnrd-apps.bb` 中引用

1. 修改现有开源组件并生成板级 Patch

以修改 `webui-vue` 前端界面为例：

(1) 提取源码到工作区 (`devtool modify`)

```
cd ~/openbmc/build/ceb-gnrd  
devtool modify webui-vue
```

Yocto 会自动将 `webui-vue` 源码提取到 `build/ceb-gnrd/workspace/sources/webui-vue/` 并初始化为 Git 仓库。

(2) 修改代码并提交

```
cd workspace/sources/webui-vue  
# 进行代码修改 ...  
  
# 检查修改状态并提交  
git status  
git diff  
git add .  
git commit -m "ceb-gnrd: Customize webui theme  
  
Signed-off-by: Your Name <your.email@example.com>"
```

(3) 实时编译与验证

```
cd ~/openbmc/build/ceb-gnrd
devtool build webui-vue

# 或构建完整镜像并在 QEMU 中验证
devtool build-image obmc-phosphor-image
```

(4) 生成 Patch 并归档至板级目录 (`devtool finish`)

```
devtool finish webui-vue ../meta-ctopai/meta-ceb-gnrd
```

- 执行结果:

1. 自动在 `meta-ctopai/meta-ceb-gnrd/recipes-phosphor/webui/files/` 目录下生成标准的 `000x-ceb-gnrd-*.patch`。
2. 自动在 `meta-ctopai/meta-ceb-gnrd/recipes-phosphor/webui/webui-vue_%.bbappend` 的 `SRC_URI` 中追加该补丁引用。
3. 自动从 `workspace` 中清理开发状态 (无需再手动执行 `reset`)。

2. 向现有组件中增加全新源代码文件 / 模块

当需要为现有项目 (如 `bmcweb` 或 `dbus-sensors`) 新增自定义 `.cpp`、`.hpp` 或网页组件时:

(1) 进入源码工作区新增文件

```
cd ~/openbmc/build/ceb-gnrd
devtool modify dbus-sensors
cd workspace/sources/dbus-sensors
```

(2) 创建新文件并注册构建系统

```
# 创建新的传感器实现文件
touch src/CebGnrdSpecialSensor.cpp
touch src/CebGnrdSpecialSensor.hpp

# 在 CMakeLists.txt 或 meson.build 中将新文件加入编译列表
```

(3) 将新文件纳入 Git 管理并提交

```
git add src/CebGnrdSpecialSensor.cpp src/CebGnrdSpecialSensor.hpp CMakeLists.txt
git commit -m "ceb-gnrd: Add special sensor monitoring module

Signed-off-by: Your Name <your.email@example.com>"
```

(4) 编译并输出至板卡层

```
cd ~/openbmc/build/ceb-gnrd
devtool build dbus-sensors
devtool finish dbus-sensors ../meta-ctopai/meta-ceb-gnrd
```

devtool 会将包含新增代码文件的 Git Commit 完整转换为包含新建文件的 Git Patch，并自动保存到板卡级目录的 `files/` 中。

3. 使用 `devtool add` 创建全新的独立软件包 / 配方

当需要为 `ceb-gnrd` 引入一个全新的板级私有工具、自研服务或第三方库时：

(1) 基于本地源码或远程仓库新建配方

- 方式 A：基于本地源码目录：

```
cd ~/openbmc/build/ceb-gnrd
devtool add ceb-gnrd-tool /home/test/src/ceb-gnrd-tool
```

- 方式 B：基于远程 Git 仓库：

```
devtool add my-custom-app https://github.com/my-org/my-custom-app.git
```

devtool 会自动检测源码结构（如 CMake、Meson、Autotools、Python、Makefile），并在 `workspace/recipes/` 自动生成对应的 `.bb` 配方文件。

(2) 编译与调试配方

```
devtool build ceb-gnrd-tool
```

如果需要微调配方逻辑（如追加依赖、修改安装路径），可直接编辑 `workspace/recipes/ceb-gnrd-tool/ceb-gnrd-tool.bb`。

(3) 归档全新配方到板卡层

```
devtool finish ceb-gnrd-tool ../meta-ctopai/meta-ceb-gnrd
```

执行后，该新配方会自动安装在 `meta-ctopai/meta-ceb-gnrd/recipes-ceb-gnrd/ceb-gnrd-tool/ceb-gnrd-tool.bb`。

(4) 将新软件包打包加入板卡镜像

在 `meta-ctopai/meta-ceb-gnrd/recipes-phosphor/packagegroups/packagegroup-ceb-gnrd-apps.bb` 中加入：

```
RDEPENDS:${PN}-system += " \
    ceb-gnrd-tool \
    "
```

或在 `meta-ctopai/meta-ceb-gnrd/recipes-phosphor/images/obmc-phosphor-image.bbappend` 中追加：

```
IMAGE_INSTALL:append = " ceb-gnrd-tool"
```

4. `devtool update-recipe` 与 `devtool finish` 的核心区别与配合

对比维度	<code>devtool update-recipe</code>	<code>devtool finish</code>
一句话定位	阶段性更新（保留开发状态）	最终交付并退出（完成开发）
生成 Patch / 更新配方	✅ 会将 Git 提交生成 Patch 并更新至指定图层	✅ 会将 Git 提交生成 Patch 并更新至指定图层
工作区状态 (workspace)	🟢 继续保留，处于开发模式	🔴 自动清理并删除 workspace 中的源码和临时配方
后续 bitbake 编译来源	依然优先使用 <code>workspace/sources/</code> 下的代码	回归标准流程（从上游下载解压 + 板卡层 Patch 打补丁）
适用场景	边开发边同步 Patch、需要继续调试修改	功能完全开发、测试通过，准备提交代码归档

⚠️ 常见疑问：`devtool finish` 执行完后还需要执行 `devtool reset` 吗？

答案：完全不需要！
因为 `devtool finish` 的底层逻辑已经自动包含了 `reset` 清理机制。执行成功后，`build/ceb-gnrd/workspace/sources/<recipe>` 目录和临时配置已被自动清除，该组件已彻底退出开发模式。如果此时再执行 `devtool reset`，系统会报错提示该配方不在工作区中。

只有在主动放弃当前修改、中途丢弃所有未保存改动时，才需要手动执行 `devtool reset <recipe>`（若有未提交的改动可加 `-f` 强制放弃：`devtool reset -f <recipe>`）。

💡 推荐协同工作流：

```
# 1. 提取源码进入开发模式
devtool modify bmcweb

# 2. 修改代码并提交
cd workspace/sources/bmcweb
git commit -am "ceb-gnrd: Add custom redfish OEM handler"

# 3. 编译验证
devtool build bmcweb

# 4. [阶段性保存] 临时同步 Patch 到板卡层，不退出开发（必须加 -a 指定板卡层）
devtool update-recipe -a ../meta-ctopai/meta-ceb-gnrd bmcweb

# 5. [最终交付] 全部功能测试通过后，归档 Patch 并彻底清理工作区（会自动清理 workspace）
devtool finish bmcweb ../meta-ctopai/meta-ceb-gnrd
```

5. devtool 其他实用命令速查

操作需求	执行命令
仅更新补丁不退出工作区	<code>devtool update-recipe -a ../meta-ctopai/meta-ceb-gnrd <recipe></code>
构建包含工作区代码的完整镜像	<code>devtool build-image obmc-phosphor-image</code>
放弃修改/重置工作区	<code>devtool reset <recipe></code> (强制放弃加 <code>-f</code>)
查看当前处于工作区的配方列表	<code>devtool status</code>